Comparación empirica de los algoritmos de ordenamiento Inserción y Quicksort: tiempo, comparaciones y profundidad de recursión

Title (EN): Experimental comparison of the Insertion Sort and Quicksort algorithms: running time, comparisons and recursion depth

Autor: Rodrigo Kenny Calcina Uribe

Filiación: Escuela Profesional de Ingeniería en Informática y Sistemas, Facultad de Ingeniería, Universidad Nacional Jorge Basadre Grohmann, Tacna, Perú.

Correo institucional: rcalcinau@unjbg.edu.pe

Resumen

Se comparó empíricamente el ordenamiento por Inserción (método directo, $O(n^2)$) con Quicksort (método logarítmico, O(n log n) en promedio) para determinar cómo cambian su tiempo de ejecución, sus comparaciones, sus intercambios y su profundidad de recursión según el tamaño y el patrón de la entrada. Ambos algoritmos se implementaron en C++ y se midieron con `std::chrono::steady_clock` sobre cinco patrones (aleatorio, ascendente, descendente, casi ordenado y con duplicados), tres tamaños (n = 1 000, 10 000 y 100 000) y 30 repeticiones por condición, con semillas fijas y el mismo equipo. Con n = 100 000 y entrada aleatoria, el tiempo medio fue de 3 105 ms en Inserción y de 15.3 ms en Quicksort (unas 200 veces más); con entrada descendente, la diferencia fue de unas 2 000 veces. Solo con entrada ascendente Inserción fue más rápida (0.32 ms frente a 2.70 ms). Al multiplicar n por 10, las comparaciones de Inserción se multiplicaron por 100 y las de Quicksort por cerca de 12, como predicen $O(n^2)$ y O(n log n). Se concluye que Quicksort con mediana de tres es preferible en general, e Inserción solo conviene con datos ya ordenados.

Palabras clave: algoritmos de ordenamiento; Inserción; Quicksort; complejidad computacional; evaluación experimental.

Abstract

Insertion Sort (a direct method, $O(n^2)$) was empirically compared with Quicksort (a logarithmic method, O(n log n) on average) to determine how running time, comparisons, swaps and recursion depth change with input size and input pattern. Both algorithms were implemented in C++ and measured with **`std::chrono::steady_clock`** on five input patterns (random, ascending, descending, nearly sorted and with duplicates), three sizes (n = 1,000, 10,000 and 100,000) and 30 repetitions per condition, using fixed seeds and the same machine. Mean, median, standard deviation, minimum and maximum running times were computed. With n = 100,000 and random input, the mean time was 3,105 ms for Insertion Sort and 15.3 ms for Quicksort (about 200 times more); with descending input, the gap was about 2,000 times. Only with ascending input

was Insertion Sort faster (0.32 ms versus 2.70 ms). When n was multiplied by 10, the comparisons of Insertion Sort grew 100 times and those of Quicksort about 12 times, as predicted by $O(n^2)$ and $O(n \log n)$. It is concluded that Quicksort with median-of-three is preferable in general, and Insertion Sort is only convenient with already sorted data.

Keywords: sorting algorithms; Insertion Sort; Quicksort; computational complexity; experimental evaluation.

1. Introducción

Ordenar datos es una de las operaciones más frecuentes en el software: la búsqueda eficiente, la eliminación de duplicados y el análisis de datos suelen depender de que los elementos estén ordenados. Por eso los algoritmos de ordenamiento son un caso clásico para estudiar la complejidad computacional, ya que existen métodos simples pero costosos y métodos más elaborados pero más eficientes (Cormen et al., 2022).

La complejidad asintótica describe cómo crece el costo de un algoritmo con el tamaño de la entrada, pero no siempre predice el comportamiento medido, que además depende del patrón de los datos, de la implementación y del equipo. Por ello, la literatura incluye estudios comparativos que miden algoritmos de ordenamiento de forma empírica (Ali et al., 2018; Frak et al., 2018; García y Gonzáles, 2023; Mamatnabiyev y Zhaparov, 2021; Murillo y Caamaño, 2013; Salgado et al., 2022). En esta comparación se han empleado tanto insercion como quicksort para ver la complejidad y diferencia entre ellos.

En este trabajo se comparan dos algoritmos representativos: el ordenamiento por Inserción, un método directo de complejidad cuadrática, y Quicksort (Hoare, 1962), un método basado en dividir y vencer con complejidad promedio $O(n \log n)$. La comparación se hace sobre cinco patrones de entrada, tres tamaños y 30 repeticiones por condición, y se registran el tiempo, las comparaciones, los intercambios y la profundidad de recursión.

Objetivo. Comparar empíricamente Inserción y Quicksort y relacionar los resultados con su complejidad teórica y su estabilidad.

Hipótesis descriptivas. Estas expectativas surgen de la teoría y se contrastan con lo observado, sin pruebas inferenciales:

- H1. Con entradas aleatorias grandes, Quicksort será más rápido que Inserción y la diferencia aumentará con n.
- H2. Con entrada ascendente, Inserción se comportará de forma casi lineal y podrá igualar o superar a Quicksort.
- H3. Con entrada descendente, Inserción tendrá su peor caso y será el patrón más costoso para este algoritmo.
- H4. Con la variante de Quicksort usada (pivote por mediana de tres), los patrones ordenados no producirán su peor caso cuadrático ni una recursión profunda.

2. Método

2.1 Algoritmos y variantes

Se implementaron versiones propias en C++, sin usar std::sort, con contadores de comparaciones, intercambios y profundidad de recursión.

Inserción. Recorre el arreglo y desplaza hacia la derecha los elementos mayores que el elemento actual hasta encontrar su posición. Es estable y trabaja in situ. Cada comparación entre un elemento y la clave se cuenta como una comparación; cada desplazamiento se cuenta como un intercambio (en rigor es un movimiento, no un intercambio de dos posiciones).

Quicksort. Versión recursiva con pivote por **mediana de tres** (primer, central y último elemento) y **partición de Hoare**. Esta variante evita el peor caso con entradas ordenadas y se comporta bien con muchos valores repetidos. Es inestable. La mediana de tres se contabilizó como 3 comparaciones por llamada, y la profundidad de recursión se midió como el nivel máximo de llamadas anidadas, con la primera llamada en el nivel 1. Se compara así una versión optimizada de Quicksort con una versión canónica de Inserción; esta decisión se declara porque el pivote ingenuo cambiaría de forma notable los resultados con datos ordenados.

2.2 Complejidad teórica

Para Inserción, en el peor caso (arreglo descendente) el elemento en la posición i se compara con los i elementos anteriores, por lo que el número de comparaciones es

$\Sigma\, i$, para $i = 1, \ldots, n - 1 = n(n - 1)/2 \leq c \cdot n^2$, es decir, $O(n^2)$.

En el mejor caso (arreglo ascendente) se hace una sola comparación por elemento, $n - 1$ en total, lo que da $O(n)$. En el caso promedio, con datos aleatorios, el número de comparaciones es del orden de $n^2/4$, o sea $O(n^2)$. El uso de memoria adicional es $O(1)$.

Para Quicksort, si cada partición divide el arreglo en partes de tamaño similar, el costo cumple $T(n) = 2T(n/2) + c \cdot n$, que resuelve a $O(n \log n)$. Si la partición es muy desbalanceada, $T(n) = T(n - 1) + c \cdot n$, que resuelve a $O(n^2)$. La mediana de tres reduce la probabilidad de este peor caso con entradas ordenadas. La memoria adicional es la pila de recursión, esperada $O(\log n)$.

Algoritmo	Mejor caso	Caso promedio	Peor caso	Estable	Memoria adicional
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	Sí	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	$O(\log n)$ esperada

2.3 Generación de datos

Los arreglos son de enteros y se generaron con el generador `mt19937` de C++. Se usaron cinco patrones:

1. **Aleatorio:** enteros uniformes entre 0 y 1 000 000.
2. **Ascendente:** 0, 1, …, n − 1.

3. **Descendente:** n, n − 1, …, 1.
4. **Casi ordenado:** arreglo ascendente al que se le aplican n/20 intercambios aleatorios entre pares de posiciones.
5. **Con duplicados:** enteros uniformes entre 0 y 14 (15 valores únicos repetidos).

La semilla base fue 12345 y en la repetición r se usó la semilla 12345 + r. En cada repetición ambos algoritmos recibieron una copia del mismo arreglo. Los patrones ascendente y descendente no dependen de la semilla, por lo que su variación entre repeticiones proviene solo del ruido de la medición.

2.4 Tamaños, repeticiones y métricas

Se usaron n = 1 000, 10 000 y 100 000 con 30 repeticiones por condición (algoritmo × patrón × tamaño), lo que da 2 × 5 × 3 × 30 = 900 ejecuciones. Para cada ejecución se registró:

- el tiempo en milisegundos con `std::chrono::steady_clock`, midiendo solo el ordenamiento y excluyendo la copia del arreglo, la verificación y la escritura a disco;
- el número de comparaciones y de intercambios, con contadores locales;
- la profundidad máxima de recursión (solo Quicksort; en Inserción es 0);
- la semilla y el hash del commit del código.

Tras cada ordenamiento se verificó que el arreglo resultara ordenado.

2.5 Entorno experimental

Todas las ejecuciones se realizaron en el mismo equipo, conectado a la corriente y sin otros programas pesados abiertos.

Elemento	Valor
CPU	Intel(R) Core(TM) i3-6006U, 2 núcleos / 4 procesadores lógicos, frecuencia base de 2.00 GHz, caché L3 de 3 MB
RAM	11.9 GB
Sistema operativo	Windows 10 Home Single Language, versión 22H2 (compilación 19045.6466)
Compilador	g++ (Rev8, Built by MSYS2 project) 15.2.0
Opciones de compilación	`-O2 -std=c++17`
Código y datos	https://github.com/kennym7/producto-ordenamiento (commit `7b3eca4`)

2.6 Análisis estadístico

Para cada combinación (algoritmo, patrón, n) se calcularon la media, la mediana, la desviación estándar muestral (n − 1), el mínimo y el máximo del tiempo de las 30 repeticiones, además de los promedios de comparaciones e intercambios y la profundidad máxima de recursión.

Las estadísticas, las tablas y las figuras se obtuvieron con un script auxiliar en Python 3.14.3 (analisis/analisis.py, con las bibliotecas pandas y matplotlib) a partir de results/benchmark.csv. El script agrupa las 30 repeticiones de cada condición, calcula las estadísticas descriptivas y genera las Tablas 1 y 2 y las Figuras 1 y 2. Se ejecuta con python analisis/analisis.py results/benchmark.csv; el README del repositorio explica cómo reproducirlo. En las figuras se usó escala logarítmica porque los tiempos de ambos algoritmos difieren en varios órdenes de magnitud.

3. Resultados

Los resultados provienen de results/benchmark.csv (900 ejecuciones). La estadística es descriptiva.

3.1 Tiempo de ejecución

Tabla 1. Tiempo de ejecución (ms) con n = 100 000 y 30 repeticiones por condición. Fuente: elaboración propia a partir de results/benchmark.csv.

Patrón	Algoritmo	Media	Mediana	Desv. est.	Mín.	Máx.
Aleatorio	Inserción	3105.065	3144.330	120.718	2915.460	3297.090
Aleatorio	Quicksort	15.347	14.161	3.182	13.872	29.956
Ascendente	Inserción	0.318	0.301	0.043	0.301	0.474
Ascendente	Quicksort	2.697	2.654	0.180	2.630	3.627
Descendente	Inserción	6271.632	6178.515	201.750	6091.690	6895.320
Descendente	Quicksort	3.004	2.714	1.053	2.674	8.377
Casi ordenado	Inserción	396.827	376.663	51.473	353.135	569.394
Casi ordenado	Quicksort	4.916	4.340	1.448	4.154	11.158
Duplicados	Inserción	2923.731	2910.535	241.970	2702.650	3706.820
Duplicados	Quicksort	6.004	5.766	0.598	5.508	7.772

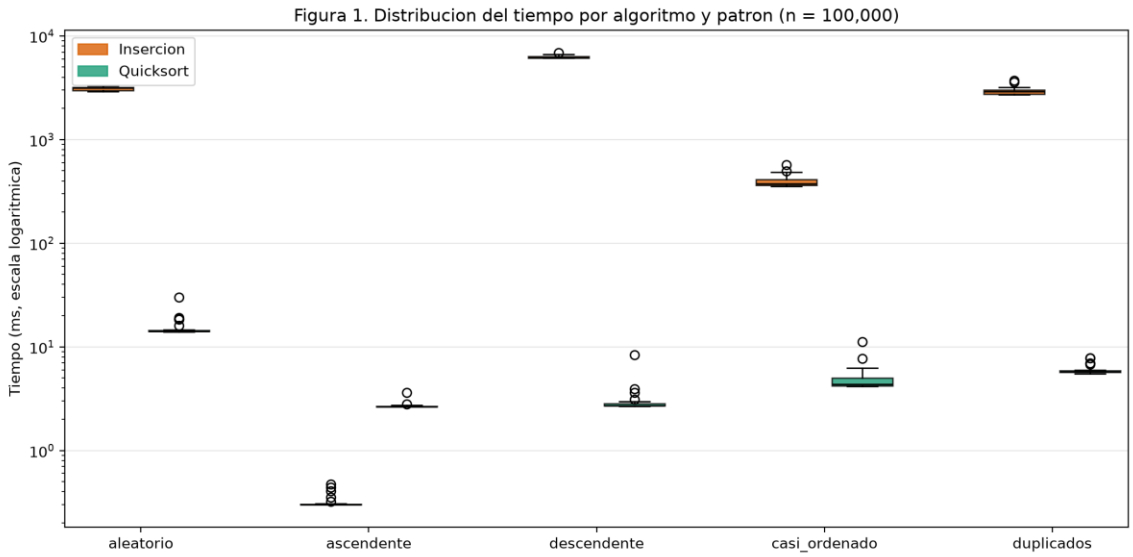


Figura 1. Distribución del tiempo por algoritmo y patrón con n = 100 000 (escala logarítmica). Fuente: elaboración propia a partir de `results/benchmark.csv`.

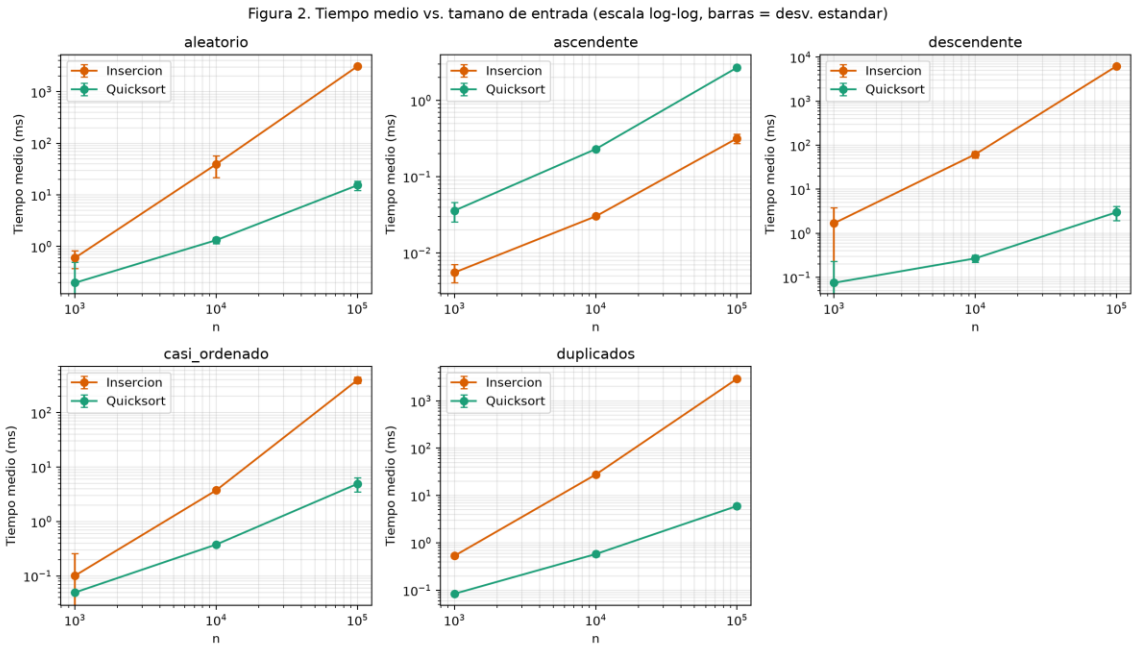


Figura 2. Tiempo medio frente al tamaño de entrada n para cada patrón (escala log-log; las barras indican la desviación estándar). Fuente: elaboración propia a partir de `results/benchmark.csv`.

Con n = 100 000, Quicksort fue más rápido que Inserción en cuatro de los cinco patrones (Tabla 1 y Figura 1). Con entrada aleatoria, el tiempo medio de Inserción fue de 3 105 ms frente a 15.3 ms de Quicksort, es decir, unas 202 veces más. La diferencia fue aún mayor con entrada descendente (6 272 ms contra 3.0 ms, unas 2 090 veces) y con duplicados (2 924 ms contra 6.0 ms, unas 490 veces); con entrada casi ordenada fue de unas 81 veces (397 ms

contra 4.9 ms). La única excepción fue la entrada ascendente: Inserción necesitó 0.32 ms y Quicksort 2.70 ms, de modo que Inserción fue unas 8.5 veces más rápida.

El tiempo de Inserción fue el más alto con entrada descendente (6 272 ms), seguido del patrón aleatorio (3 105 ms) y del de duplicados (2 924 ms). En Quicksort, el patrón más lento fue el aleatorio (15.3 ms) y el más rápido el ascendente (2.7 ms).

La variación entre repeticiones fue pequeña en Inserción con entradas aleatorias y descendentes (la desviación estándar fue el 3.9 % y el 3.2 % de la media) y mayor en Quicksort con entrada descendente (35 %) y casi ordenada (29.5 %). En Quicksort, la media fue mayor que la mediana en los cinco patrones y la Figura 1 muestra valores atípicos altos (por ejemplo, un máximo de 29.96 ms frente a una mediana de 14.16 ms con entrada aleatoria), lo que sugiere que algunas repeticiones se vieron afectadas por el ruido del sistema.

La Figura 2 muestra cómo creció el tiempo medio al aumentar n. Al pasar de n = 10 000 a n = 100 000, el tiempo de Inserción se multiplicó por 79 con entrada aleatoria, por 101 con descendente, por 105 con duplicados y por 106 con casi ordenada; con entrada ascendente se multiplicó por 10.5. En Quicksort, el mismo paso multiplicó el tiempo por valores entre 10.4 y 13.0 en todos los patrones.

3.2 Comparaciones e intercambios

Tabla 2. Comparaciones, intercambios y profundidad máxima de recursión con n = 100 000 (promedio de 30 repeticiones; la profundidad es la máxima observada). Fuente: elaboración propia a partir de **results/benchmark.csv.**

Patrón	Algoritmo	Comparaciones	Intercambios	Profundidad máx.
Aleatorio	Inserción	2 499 714 847	2 499 614 859	0
Aleatorio	Quicksort	2 451 619	397 543	38
Ascendente	Inserción	99 999	0	0
Ascendente	Quicksort	2 068 924	0	18
Descendente	Inserción	4 999 950 000	4 999 950 000	0
Descendente	Quicksort	2 068 925	50 000	18
Casi ordenado	Inserción	309 575 945	309 475 946	0
Casi ordenado	Quicksort	2 133 294	30 974	28
Duplicados	Inserción	2 333 006 799	2 332 906 803	0
Duplicados	Quicksort	2 215 486	712 115	27

Las comparaciones de Inserción coincidieron con los valores teóricos (Tabla 2). Con entrada descendente fueron 4 999 950 000, exactamente $n(n-1)/2$; con entrada ascendente, 99 999, es decir, $n-1$; y con entrada aleatoria, 2 499 714 847, muy cerca de $n^2/4 = 2\,500\,000\,000$. Al

multiplicar n por 10, las comparaciones de Inserción se multiplicaron por cerca de 100 en todos los patrones salvo el ascendente, donde se multiplicaron por 10. Las de Quicksort se multiplicaron por valores entre 11.8 y 12.6.

Con n = 100 000, Quicksort hizo entre 2.07 y 2.45 millones de comparaciones según el patrón, es decir, entre 1.25 y 1.48 veces $n \cdot \log_2 n$ (que vale unos 1.66 millones). Con entrada ascendente, Inserción hizo unas 21 veces menos comparaciones que Quicksort (99 999 contra 2 068 924).

En cuanto a los intercambios, Inserción no hizo ningún desplazamiento con entrada ascendente y, en los demás patrones, hizo casi tantos como comparaciones (con entrada descendente, exactamente los mismos: 4 999 950 000). Quicksort hizo muchos menos: 397 543 con entrada aleatoria, 712 115 con duplicados, 30 974 con casi ordenada, 50 000 con descendente (n/2) y ninguno con ascendente. Estos valores no son directamente comparables entre algoritmos, porque en Inserción cada intercambio es un desplazamiento y en Quicksort es un intercambio de dos posiciones.

3.3 Profundidad de recursión

En Inserción la profundidad de recursión es 0, porque no es un algoritmo recursivo. En Quicksort, con n = 100 000, la profundidad máxima fue de 38 niveles con entrada aleatoria, 28 con casi ordenada, 27 con duplicados y 18 con entradas ascendente y descendente (Tabla 2). Al pasar de n = 1 000 a n = 100 000, la profundidad creció de 23 a 38 (aleatorio), de 17 a 28 (casi ordenado), de 20 a 27 (duplicados) y de 11 a 18 (ascendente y descendente). Es un aumento lento, compatible con un crecimiento logarítmico: con entradas ordenadas coincide con $\log_2 n$ redondeado hacia arriba más 1 (11, 15 y 18 niveles para n = 1 000, 10 000 y 100 000), y con entrada aleatoria fue de unas 2.3 veces $\log_2 n$ con n = 100 000.

4. Discusión

Contraste con la complejidad teórica. La teoría anticipa que Inserción crece de forma cuadrática y Quicksort cerca de n log n. Los datos lo respaldan sobre todo en las comparaciones, que no dependen del equipo: las de Inserción se multiplicaron por cerca de 100 al multiplicar n por 10 (como en n^2) y las de Quicksort por unas 12 (como en n log n, que da cerca de 12.5 para ese paso). En el tiempo, el comportamiento fue parecido pero con más ruido. Con entrada aleatoria, Inserción multiplicó su tiempo por 65 (de 1 000 a 10 000) y por 79 (de 10 000 a 100 000), por debajo de 100; una posible explicación es que con n pequeño los tiempos son de fracciones de milisegundo y el ruido de la medición pesa más, aunque este experimento no lo comprobó. Con entrada descendente, casi ordenada y con duplicados, el último paso sí fue cercano a 100 (101, 106 y 105). Quicksort multiplicó su tiempo por entre 10.4 y 13.0 de 10 000 a 100 000, consistente con n log n.

Efecto del patrón de entrada. H1 se cumplió: con entrada aleatoria, Quicksort fue 3.0, 29.7 y 202 veces más rápido que Inserción para n = 1 000, 10 000 y 100 000, es decir, la diferencia aumentó con n. H2 también se cumplió, e incluso con más ventaja de la esperada: con entrada ascendente Inserción hizo solo n − 1 comparaciones (99 999) y fue unas 8.5 veces más rápida que Quicksort, que hizo cerca de 21 veces más comparaciones. H3 se cumplió: la entrada

descendente fue el peor caso de Inserción (6 272 ms y $n(n-1)/2$ comparaciones), con el doble de tiempo que la aleatoria (2.02 veces), como se espera al comparar $n(n-1)/2$ con $n^2/4$. H4 se cumplió: con la mediana de tres, Quicksort no tuvo comportamiento cuadrático con entradas ordenadas (2.7 ms y 3.0 ms, con profundidad 18).

Un resultado que los datos no explican por sí solos es que Quicksort fue entre 5 y 6 veces más rápido con entradas ordenadas (2.7 y 3.0 ms) que con la aleatoria (15.3 ms), aunque la diferencia en comparaciones fue pequeña (2.07 millones frente a 2.45 millones). Una posible explicación, que no se verificó, es que el procesador predice mejor las comparaciones cuando los datos siguen un orden regular. Comprobarlo requeriría herramientas de medición que no se usaron aquí.

Estabilidad y duplicados. Inserción es estable y Quicksort no lo es: ante elementos iguales, Quicksort puede alterar su orden relativo original. En este experimento los elementos son enteros sin información adicional, por lo que la inestabilidad no cambia el resultado visible (dos enteros iguales son indistinguibles); solo importaría si se ordenaran registros por una clave con valores repetidos. Lo que sí puede afectar es el rendimiento, y los datos muestran lo siguiente.

Con solo 15 valores únicos, Quicksort no se degradó: con n = 100 000 tardó 6.0 ms, menos que con entrada aleatoria (15.3 ms), con 2.22 millones de comparaciones, parecidas a las de los demás patrones, y profundidad 27. Sus intercambios sí aumentaron (712 115 contra 397 543 en el patrón aleatorio), algo esperable porque la partición de Hoare también intercambia elementos iguales al pivote. En Inserción, el patrón con duplicados fue algo más rápido que el aleatorio (2 924 ms contra 3 105 ms) y tuvo un 6.7 % menos de comparaciones (2 333 millones contra 2 500 millones). Esto se debe a que la comparación es estricta (a[j] > clave): Inserción no desplaza elementos iguales, así que cada elemento se detiene antes cuando encuentra uno igual.

Limitaciones y amenazas a la validez.

- Los contadores se actualizan dentro del intervalo medido, lo que añade un pequeño costo al tiempo de ambos algoritmos.
- El tiempo se midió en un solo equipo y sistema operativo, con ruido propio de un sistema multitarea; los resultados absolutos no son generalizables a otros equipos.
- La recursión de Quicksort depende del tamaño de pila; con la mediana de tres no hubo riesgo en las entradas usadas, pero un pivote ingenuo con datos ordenados podría desbordar la pila con n = 100 000.
- Los patrones ascendente y descendente son deterministas, por lo que sus 30 repeticiones solo reflejan variabilidad de la medición.
- El patrón casi ordenado aplica n/20 intercambios aleatorios, por lo que altera como máximo el 10 % de las posiciones, y no exactamente el 5 %.
- Se compara una versión optimizada de Quicksort con una versión canónica de Inserción; otras variantes (por ejemplo, Inserción con búsqueda binaria o Quicksort con otro pivote) podrían dar resultados distintos.
- Se usaron solo enteros; con otros tipos de dato, la comparación tendría otro costo.

5. Conclusiones

Los resultados respaldan las hipótesis H1 a H4 y coinciden con la complejidad teórica de ambos algoritmos.

1. Con entradas aleatorias, Quicksort fue mucho más rápido que Inserción: 15.3 ms contra 3 105 ms con n = 100 000 (unas 200 veces), y la diferencia creció con el tamaño (3, 30 y 200 veces para n = 1 000, 10 000 y 100 000).
2. Inserción tuvo su mejor caso con entrada ascendente (99 999 comparaciones y 0.32 ms, unas 8.5 veces más rápida que Quicksort) y su peor caso con entrada descendente (n(n − 1)/2 comparaciones y 6 272 ms).
3. Las comparaciones siguieron la teoría: al multiplicar n por 10, las de Inserción se multiplicaron por cerca de 100 y las de Quicksort por entre 11.8 y 12.6. La profundidad de recursión de Quicksort creció lentamente, hasta 38 niveles con n = 100 000.
4. Con 15 valores únicos, Quicksort con partición de Hoare no se degradó (6.0 ms). Su inestabilidad no cambia el resultado con enteros simples, pero importaría al ordenar registros.
5. Para estas variantes y este equipo, Quicksort con mediana de tres es la mejor opción en general, e Inserción solo conviene con datos ya ordenados. Los resultados corresponden a un solo equipo y a arreglos de enteros.

Referencias

Ali, I., Nawaz, H., Khan, I., Marilo, A., Ameen, M., & Malook, M. (2018). Performance Comparison between Merge and Quick Sort Algorithms in Data Structure. *International Journal of Advanced Computer Science and Applications, 9*(11). https://doi.org/10.14569/ijacsa.2018.091127

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Frak, A. N., Saringat, M. Z., Prasetyo, Y. A., Mustapha, A., Aman, H., & Ibrahim, N. (2018). Comparison study of sorting techniques in static data structure. https://journal.uthm.edu.my/index.php/ijie/article/view/2836

García, D., & Gonzáles, D. (2023). Comparación empírica del número de operaciones de diferentes algoritmos de ordenamiento. https://ri.uaemex.mx/handle/20.500.11799/139241

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal, 5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Mamatnabiyev, Z., & Zhaparov, M. (2021). Comparative analysis of sorting algorithms used in competitive programming. *SDU Bulletin: Natural and Technical Sciences, 55*(2), 64–70. https://journals.sdu.edu.kz/index.php/nts/article/view/546

Murillo, J., & Caamaño, S. (2013). Comparación entre algoritmos recursivos e iterativos y su medición en términos de eficiencia. *Uniciencia, 27*(1), 341–350. https://www.redalyc.org/pdf/4759/475947762020.pdf

Salgado, M., Pérez, J., & Albarrán, S. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia E Ingeniería, 43*(03), 273–280. https://doi.org/10.53766/cei/2022.43.03.05